

HDI Predictor Documentation

Code-Layout, Readability and Reusability

Date	02 July 2025
Team ID	SB-GENAI-HDI-2025
Project Name	HDI Predictor - Human Development Index Classification System
Prepared By	Kankipati Sai Venkat

Code Layout Checklist

S.No	Code Quality Parameter	Description	Followed (Yes/No/Partial)	Remarks
1	Consistent Indentation	Uniform 4-space indentation used throughout	Yes	PEP8-compliant
2	Proper File Structure	data/, notebooks/, model/, app/, templates/, Dockerfile logically separated	Yes	Mirrors standard Flask ML-app layout
3	Meaningful Variable Names	Feature and function names reflect their purpose (e.g., life_expectancy, predict_hdi_tier)	Yes	—
4	Function / Method Names	Functions are descriptively named (load_model, preprocess_input, predict)	Yes	—
5	Code Comments	Inline comments explain preprocessing steps and model choices	Yes	Docstrings added to core functions
6	Modular Design	Preprocessing, model loading, and prediction logic split into separate functions/modules	Yes	Keeps app.py thin
7	No Redundant Code	Duplicate scaling/validation logic consolidated into shared helper functions	Yes	—
8	Error Handling	Invalid or missing form inputs return	Yes	try/except around inference call

		a friendly error instead of crashing		
--	--	---	--	--

Reusable Components / Modules

S.No	Component / Module Name	Language / Technology	Reusability Level	Where Reused
1	preprocess_input()	Python / pandas	High	Used by both the Flask route and standalone batch-scoring scripts
2	load_model()	Python / joblib	High	Reused across the web app and evaluation notebook
3	predict_hdi_tier()	Python / scikit-learn	High	Core inference function reused for single and batch predictions
4	Input validation helpers	Python	Medium	Shared between the web form handler and unit tests
5	Dockerfile / requirements.txt	Docker	High	Reusable base for any future scikit-learn + Flask service

Overall Code Quality Assessment

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Clear separation between data, model, and app layers
Readability	5	Descriptive names and consistent style throughout
Reusability	4	Core preprocessing/inference functions are cleanly reusable
Documentation / Comments	4	Docstrings present on all core functions; README covers setup
Overall Score	4.5 / 5	Solid, maintainable codebase ready for extension